Universidad Distrital Francisco José de Caldas
Facultad de Ingeniería – Ingeniería de Sistemas

Taller para la Casa – Antipatrones

La Oficina de Visas del Caos (Java)

Asignatura: Modelos de Programación
Docente: Bocanegra

Integrantes
Eddie Santiago Rondón Capera – 20251020108
Luna Alejandra Sandoval Rodríguez – 20241020053

Bogotá D.C., noviembre de 2026

Taller para la Casa – La Oficina de Visas del Caos

Tema: Antipatrones de diseño de software y cómo salir de ellos con patrones

Modalidad: Trabajo individual o en parejas

Objetivo del taller

Reconocer cuatro antipatrones clásicos (God Object, Copy-Paste Programming, Magic Numbers y Acoplamiento fuerte) dentro de un sistema heredado, explicar el daño que causa cada uno y refactorizarlo en Java con un patrón o buena práctica vista en clase, demostrando en consola que la versión nueva se porta mejor que la vieja.

Situación propuesta

El Consulado Caótico: sistema de trámite de visas entre países

El Consulado Caótico gestiona visas de turismo, estudio y trabajo para viajeros de todo el mundo. Su sistema lleva quince años funcionando a punta de parches, y el equipo de TI ya no se atreve a tocarlo. Estos son los síntomas que la gente reporta:

- Ramiro, el empleado de la ventanilla única, revisa el pasaporte, cobra la tarifa, guarda el expediente y envía el correo. Cuando Ramiro se enferma, el consulado entero se detiene.
- Calcular el precio de una visa de turismo, de estudio y de trabajo se hace con el mismo código copiado tres veces. El día que subieron la tarifa base, se corrigieron dos y la tercera quedó con el precio viejo.
- En el código aparecen cosas como tipo == 3 o descuento 0.5, y nadie sabe qué significan. Se cuenta que el 3 es la visa de trabajo, pero nadie lo ha confirmado.
- Cuando un viajero cancela su solicitud, hay que avisarle al correo, a la agenda de citas y a la caja de pagos, uno por uno y a mano. Un viernes se olvidaron de la caja, y a un viajero se le perdió el reembolso.

La misión del equipo es descubrir qué antipatrones tiene este sistema, dejar la evidencia a la vista y rescatarlo. La temática puede adaptarse a una aerolínea, una tienda virtual o un videojuego, manteniendo los mismos antipatrones.

El código heredado

Esto es lo que dejó el equipo anterior. Estudien cada fragmento antes de tocarlo.

Fragmento 1: la ventanilla de Ramiro

```
public class OficinaDeTodo {

    public void procesar(String nombre, String pasaporte, double precio) {
        if (pasaporte == null || pasaporte.length() < 6) {
            System.out.println("Pasaporte invalido");
            return;
        }
        System.out.println("Cobrando $" + precio + " a " + nombre);
        System.out.println("Guardando expediente de " + nombre);
        System.out.println("Correo para " + nombre + ": su visa va en camino");
    }
}
```

Fragmento 2: la tarifa clonada

```
public class CalculadoraDeTarifas {

    public double visaTurista(int meses) {
        double total = 50 * meses;
        total = total + 10;
        return total;
    }

    public double visaEstudiante(int meses) {
        double total = 50 * meses;
        total = total + 10;
        return total * 0.5;
    }

    public double visaTrabajo(int meses) {
        double total = 50 * meses;
        total = total + 10;
        return total * 1.5;
    }
}
```

Fragmento 3: los números misteriosos

```
public class EvaluadorMagico {

    public String evaluar(int tipo, int edad) {
        if (tipo == 1 && edad > 17) return "APROBADA";
        if (tipo == 2 && edad > 15 && edad < 31) return "APROBADA";
        if (tipo == 3 && edad > 17 && edad < 66) return "APROBADA";
        return "RECHAZADA";
    }
}
```

Fragmento 4: la cancelación en cadena

```
public class SolicitudAcoplada {

    private String estado = "ACTIVA";
    private CorreoServicio correo = new CorreoServicio();
    private CitasServicio citas = new CitasServicio();
    private PagosServicio pagos = new PagosServicio();

    public void cancelar() {
        estado = "CANCELADA";
        correo.avisarCancelacion();
        citas.liberarCupo();
    }
}
```

Restricción de calidad: cero antipatrones en la solución

Los fragmentos de arriba existen solo para ser rescatados. La versión nueva no puede contener ningún antipatrón: nada de clases con más de una responsabilidad, nada de lógica copiada, nada de números o textos sin nombre, y nada de llamadas manuales en cadena entre servicios. Si mañana sube una tarifa, aparece un nuevo tipo de visa o se agrega un aviso más, el cambio debe hacerse en un solo lugar sin tocar código ajeno. Prohibido salir de este taller con un nuevo Ramiro.

Actividades y casos de prueba

- **Parte 1. Cacería de antipatrones:** para cada fragmento, indicar el nombre del antipatrón, el síntoma que lo delata y la consecuencia que causa en el consulado.
- **Parte 2. El rescate:** refactorizar cada fragmento en Java, organizado por paquetes, aplicando la solución que corresponda (responsabilidad única, Strategy, enum con constantes y Observer).
- **Parte 3. La prueba de fuego:** ejecutar en Main los 5 casos siguientes, mostrando en consola la versión heredada y la versión rescatada una al lado de la otra.

Caso	Antipatrón evaluado	Entrada / Acción	Resultado esperado en consola
1	God Object	Tramitar la visa de un viajero con OficinaDeTodo y luego con las clases separadas	Ambas aprueban el trámite, pero la salida muestra quién hizo cada tarea: Ramiro lo hace todo frente a una clase por responsabilidad.
2	Copy-Paste Programming	Subir la tarifa base de 50 a 60 en la versión clonada y en la versión Strategy	En la clonada hay que editar 3 lugares (y el contador lo muestra); en Strategy, 1. El precio final coincide en ambas.
3	Magic Numbers	Evaluar una visa con tipo 2 y edad 20, y luego con TipoVisa.ESTUDIANTE; probar el tipo 9	El resultado es el mismo; el tipo 9 es rechazado en silencio en la versión heredada y con un mensaje claro en la nueva.

Caso	Antipatrón evaluado	Entrada / Acción	Resultado esperado en consola
4	Acoplamiento fuerte	Cancelar una solicitud en SolicitudAcoplada y luego en la versión con Observer	La heredada olvida el reembolso y lo deja a la vista; la nueva avisa a correo, citas y pagos sin que la solicitud los conozca.
5	Integración	Recorrer el viaje completo: solicitar, evaluar, cobrar y cancelar con la versión nueva	Todo funciona en conjunto y se imprime una tabla final con los 4 antipatrones capturados, su síntoma y su solución.

Entregable

Entregar un proyecto Java (repositorio) que compile y ejecute correctamente. La entrega debe incluir:

- Código fuente completo organizado por paquetes.
- Captura o texto de la salida en consola con los 5 casos de prueba ejecutados.
- La tabla de la Parte 1 con el antipatrón, el síntoma y la consecuencia de cada fragmento.
- Un párrafo corto explicando cómo su implementación evita caer de nuevo en los antipatrones.

Criterios de evaluación (puntaje total: 50 puntos)

Criterio	Puntos	Qué se evalúa
Cacería de antipatrones	10 (20%)	Nombra correctamente los 4 antipatrones y explica con claridad el síntoma y la consecuencia de cada uno.
Rescate: God Object y Copy-Paste	10 (20%)	Responsabilidades separadas en clases distintas y tarifas con Strategy, con un único punto de cambio del precio base.
Rescate: Magic Numbers y Acoplamiento	10 (20%)	Enum y constantes con nombre, tipos inválidos con mensaje claro, y Observer avisando a los tres servicios.
Casos de prueba y paquetes	10 (20%)	Los 5 casos compilan, se ejecutan en consola y el proyecto está bien organizado por paquetes.
Restricción de calidad	5 (10%)	La versión nueva no reproduce ningún antipatrón.
Claridad y reflexión	5 (10%)	Código legible y sencillo, buenas prácticas y reflexión breve sobre cómo evitar antipatrones.